SIT744 Assignment 4

Due: Week 11 Friday 8:00 pm (AEST)

Weight: 20% of the final mark

Mode: Individual assignment

Read the assignment instructions carefully.

What to submit

By the due date, submit the following files to the corresponding Assignment Dropbox in CloudDeakin:

- [YourID]_[UnitCode]_assignment4_solution.ipynb: your Python notebook solution source file.
- [YourID]_[UnitCode]_assignment4_output.pdf: the PDF exported from your notebook.
- **undrip_train.txt**: the training text file created in Task 1.
- **undrip_val.txt**: the validation text file created in Task 1.
- **economic_test.txt**: the held-out related evaluation text file created in Task 1.

For example, if your student ID is `123456` and you are a SIT744 student, the filenames should be:

- `123456_SIT744_assignment4_solution.ipynb`
- `123456_SIT744_assignment4_output.pdf`
- `undrip_train.txt`
- `undrip_val.txt`
- `economic_test.txt`

Please keep your answers short and to the point. Clean up code outputs to reduce unnecessary information, such as excessively long training logs.

Important instructions

- Use the provided template file `SIT744_Assignment4_student_template.ipynb` as the starting point for your notebook. If you choose not to use the template, your notebook must follow the same section order and section headings.
- In Task 1, download the two public source reports linked in this brief.
- In Task 1, create exactly three text files: `undrip_train.txt`, `undrip_val.txt`, and `economic_test.txt`.
- All later modelling tasks must use the three text files created in Task 1.
- Follow the specified preprocessing protocol. Do not invent your own split, substitute another document, or use a different evaluation text for the main results.
- Keep the assignment focused on **one language-modelling adaptation pipeline** across Tasks 1–4.
- Use the **same base model**, **same tokenizer**, **same training text**, **same validation text**, **same held-out evaluation text**, and **same generation prompt** throughout the assignment unless the task explicitly asks for a controlled change.

- For this assignment, use the fixed decoder-only model `distilgpt2`.
- For this assignment, use the fixed parameter-efficient method **LoRA**. Do not substitute another PEFT method.
- Report **perplexity** for every main evaluation. Perplexity is the fixed evaluation metric for the required results.
- All analysis and discussion must be based on your own reported results. When you make a claim about improvement, transfer, efficiency, or output quality, refer to evidence from your master results table, training-loss record, or generation comparison table. Generic discussion that is not connected to your results will receive limited credit.
- Use the fixed generation prompt exactly as written in this brief.
- There is **no fixed minimum perplexity threshold** for this assignment. Marks are based on sound setup, correct implementation, fair comparison, justified analysis, and quality of reporting.
- Extra experiments are not required and will not attract extra credit. Keep the notebook focused on the required pipeline and outputs.
- Label all tables and figures clearly.
- Stop cloud resources when they are not in use.

Warning

Some components of this assignment may involve moderate computation that runs for a noticeable duration. Start early.

Assignment objective

This assignment introduces a realistic but tightly scoped language-modelling workflow: evaluating a pretrained decoder-only model, adapting it to a specialised text domain, and comparing **full fine-tuning** against **parameter-efficient fine-tuning**.

The assignment is designed to help you consolidate both conceptual and practical skills. In particular, it is intended to push you to learn enough about:

- tokenisation for language models;
- decoder-only causal language modelling;
- perplexity as an evaluation measure;
- full fine-tuning versus LoRA;
- fair experimental comparison;
- concise reporting of model behaviour.

The goal is not to train the largest possible model or chase a single best score. The goal is to understand what changes when a pretrained language model is adapted to a narrower domain, and how much of that gain can be recovered with a lighter-weight method.

Domain and source documents

You will work with two public reports from the Joint Standing Committee on Aboriginal and Torres Strait Islander Affairs.

- **Fine-tuning source:** Inquiry into the application of the United Nations Declaration on the Rights of Indigenous Peoples in Australia
- **Held-out related evaluation source:** Inquiry into economic self-determination and opportunities for First Nations Australians

You will download these reports and convert them into plain text files for the assignment.

After preprocessing, your notebook must create the following three files:

- `undrip_train.txt`
- `undrip_val.txt`
- `economic_test.txt`

All later tasks must use these exact files. Do not use a different split or substitute another document for the main results.

Required preprocessing protocol

The data preparation step in Task 1 practises the connection between public source documents, plain text preprocessing, tokenisation, and language-model evaluation. The protocol is fixed so that later model comparisons remain fair and markable.

Follow these steps:

1. Download the two public reports from the links above. Use the Word version if available, because it is usually easier to convert cleanly to text. If only a PDF is available, convert the PDF to plain text using a suitable tool.
2. Extract the main report text into plain text.
3. Remove obvious website navigation text, repeated headers/footers, empty lines, and other artefacts where possible.
4. Keep the main report content. Minor formatting artefacts are acceptable.
5. Split the cleaned UNDRIP text into:
    - `undrip_train.txt` : first 80% of the cleaned UNDRIP text
    - `undrip_val.txt` : final 20% of the cleaned UNDRIP text
6. Use the cleaned Economic self-determination report as:
    - `economic_test.txt`
7. Save all three files in the same folder as your notebook.
8. Report the number of characters and approximate number of tokens in each file.

Suggested preprocessing scaffold

```
from pathlib import Path

def clean_text(text):
    lines = text.splitlines()
    lines = [line.strip() for line in lines]
    lines = [line for line in lines if len(line) > 0]
```

```
    return "\n".join(lines)

def split_text(text, train_ratio=0.8):
    split_index = int(len(text) * train_ratio)
    return text[:split_index], text[split_index:]

# Place your extracted raw plain text into these two files first.
undrip_raw = Path("undrip_raw.txt").read_text(encoding="utf-8")
economic_raw = Path("economic_raw.txt").read_text(encoding="utf-8")

undrip_clean = clean_text(undrip_raw)
economic_clean = clean_text(economic_raw)

undrip_train, undrip_val = split_text(undrip_clean, train_ratio=0.8)

Path("undrip_train.txt").write_text(undrip_train, encoding="utf-8")
Path("undrip_val.txt").write_text(undrip_val, encoding="utf-8")
Path("economic_test.txt").write_text(economic_clean, encoding="utf-8")
```

This scaffold supports the data preparation part of Task 1. It assumes that you have already extracted the raw report text into `undrip_raw.txt` and `economic_raw.txt`. It is not a complete PDF or Word extraction solution.

Assignment motivation

A pretrained language model already has broad linguistic knowledge, but it may not write naturally in a specialised domain such as Australian parliamentary reporting. Fine-tuning on a domain-specific text can improve its fit to that domain. However, full fine-tuning updates all model parameters, which can be more expensive. LoRA instead adds a small number of trainable parameters and keeps the original weights mostly fixed.

This assignment therefore asks a focused question:

> When adapting a small pretrained language model to a specialised parliamentary domain, how much improvement do we obtain from full fine-tuning, and how much of that improvement can LoRA recover at lower adaptation cost?

Context primer

You are expected to learn the following ideas well enough to use them responsibly in your notebook.

Decoder-only language models

A decoder-only language model predicts the next token from the tokens to its left. This is often called **causal language modelling**. GPT-style models belong to this family.

For this assignment, you are **not** training a language model from scratch. You are starting from a pretrained model and adapting it.

Tokenisation

The model does not read raw characters or words directly. It reads token IDs produced by a tokenizer. You should understand:

- why text must be tokenised before entering the model;
- that different tokenizers may split text differently;
- why you must keep the tokenizer fixed when comparing pretrained, full fine-tuned, and LoRA-adapted versions of the same model.

Perplexity

Perplexity measures how surprised a language model is by a sequence. Lower perplexity means the model assigns higher probability to the observed text.

A common form is:

$$PPL = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log p(x_i \mid x_{<i})\right)$$

You do **not** need to derive this formula in the assignment, but you should understand the practical interpretation:

- lower perplexity usually means a better fit to that text distribution;
- a large drop on the training-like domain but not on the related held-out domain can indicate narrow adaptation rather than broader transfer;
- perplexity alone does not fully capture output quality, so short qualitative inspection is also required.

Full fine-tuning

In full fine-tuning, all trainable parameters of the base model are updated on the domain text. This often improves in-domain fit, but uses more trainable parameters and can be less efficient.

LoRA

LoRA is a parameter-efficient fine-tuning method. Instead of updating every original weight, it adds a small trainable low-rank update to selected layers.

For this assignment, the main conceptual question is not the matrix derivation of LoRA, but the practical trade-off:

- fewer trainable parameters;
- lower adaptation cost;
- potentially similar, but not always identical, improvement compared with full fine-tuning.

Fair comparison

A good comparison keeps as much fixed as possible. In this assignment, fairness means:

- same base model;
- same three text files created in Task 1;
- same tokenizer;

- same evaluation protocol;
- same generation prompt;
- same master results table for quantitative comparison.

If you change too many things at once, the comparison becomes difficult to interpret.

Recommended background resources

You are encouraged to consult the following resources while working on the assignment. Learning from these materials is part of the purpose of the task.

Concepts and implementation

- Hugging Face model card: **DistilGPT2**
  https://huggingface.co/distilgpt2
- Hugging Face Transformers documentation: **Causal language modeling**
  https://huggingface.co/docs/transformers/tasks/language_modeling
- Hugging Face PEFT documentation: **PEFT overview**
  https://huggingface.co/docs/peft/index
- Hugging Face PEFT documentation: **LoRA**
  https://huggingface.co/docs/peft/package_reference/lora
- PyTorch Tutorials: **Learn the Basics**
  https://docs.pytorch.org/tutorials/beginner/basics/intro.html

Primary references

- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D. and Sutskever, I. (2019). *Language Models are Unsupervised Multitask Learners*.
  https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L. and Chen, W. (2021). *LoRA: Low-Rank Adaptation of Large Language Models*.
  https://arxiv.org/abs/2106.09685

You are **not** expected to read every detail of the research papers. The official documentation pages are the best starting point for implementation.

Assessment structure

The assignment is organised into five criteria:

- Task 1: Data preparation and pretrained baseline — 25%
- Task 2: Full fine-tuning — 25%
- Task 3: LoRA adaptation and comparison — 25%
- Task 4: Interpretation and learning reflection — 20%

- Overall notebook quality and reproducibility — 5%

Final marks are awarded using these weights and the rubric. These weights sum to 100%. One criterion covers notebook quality and reproducibility across the whole submission, while the remaining criteria align with Tasks 1–4.

Fixed protocol for this assignment

After preparing the required text files, all students must use the following fixed choices for the main modelling tasks.

Model and tokenizer

- Base model: `distilgpt2`
- Tokenizer: the tokenizer associated with `distilgpt2`

Because `distilgpt2` does not define a separate padding token by default, set `tokenizer.pad_token = tokenizer.eos_token` before batching tokenised sequences. The student template notebook includes this setup.

Text files

- Training text: `undrip_train.txt`
- Validation text: `undrip_val.txt`
- Held-out related evaluation text: `economic_test.txt`

Required generation prompt

Use this exact prompt for the generation comparison:

```
Economic self-determination for First Nations peoples requires
```

Evaluation metric

- Required metric: perplexity

Full fine-tuning protocol

Use the full fine-tuning configuration supplied in the student template notebook. Keep the base model, tokenizer, training file, validation file, held-out evaluation file, sequence length, and generation prompt fixed.

Do not conduct trial-and-error hyperparameter search for the main task. The purpose is to compare pretrained, full fine-tuned, and LoRA-adapted models under a controlled protocol.

LoRA protocol

Use the LoRA configuration provided in the student template notebook. Do not redesign LoRA for the main task.

The purpose of this assignment is to understand the comparison, not to reward trial-and-error hyperparameter search.

Task 1 Data preparation and pretrained baseline (25%)

In this task, prepare the plain text files used by the modelling tasks and establish a clean pretrained baseline before any adaptation.

You must:

- download the two source reports;
- extract plain text;
- clean the text using the required preprocessing protocol;
- create exactly `undrip_train.txt`, `undrip_val.txt`, and `economic_test.txt`;
- report basic statistics for the three files;
- evaluate pretrained `distilgpt2` perplexity on `undrip_val.txt` and `economic_test.txt`;
- complete only the pretrained row of the master results table.

Required evidence:

- the two source documents identified;
- the three generated text files;
- one short description of the extraction method used, maximum 100 words;
- one completed data-preparation table;
- the pretrained row of the master results table completed.

Use this data-preparation table:

File	Role	Character count	Approximate token count
`undrip_train.txt`	training text		
`undrip_val.txt`	validation text		
`economic_test.txt`	held-out related evaluation text		

Approximate token count may be computed using the `distilgpt2` tokenizer. Exact values may vary slightly depending on cleaning choices, but the file roles and 80/20 UNDRIP split must be followed.

Use this master results table across Tasks 1–3. In Task 1, complete only the pretrained row.

Training time depends on the runtime environment, so use it mainly for comparing your own full fine-tuning and LoRA runs under the same setup.

Model state	Parameters updated during adaptation	Training time (minutes; compare within the same runtime)	PPL on UNDRIP validation	PPL on Economic test
Pretrained `distilgpt2`	0 additional	0		
Full fine-tuned `distilgpt2`				
LoRA-adapted `distilgpt2`				

Task 2 Full fine-tuning (25%)

In this task, adapt the model using one fixed full fine-tuning run.

You must:

- fine-tune `distilgpt2` once using the full fine-tuning configuration supplied in the student template notebook;
- evaluate the full fine-tuned model on `undrip_val.txt` and `economic_test.txt`;
- complete the full fine-tuned row of the master results table;
- provide one training-loss curve, or a clearly reported final training-loss record if plotting is not feasible.

Use only the fixed full fine-tuning run supplied in the student template notebook.

Required evidence:

- the full fine-tuned row of the master results table completed;
- one training-loss curve, or one clearly reported final training-loss record.

Task 3 LoRA adaptation and comparison (25%)

In this task, compare parameter-efficient adaptation against the full fine-tuned model from Task 2 and the pretrained baseline from Task 1.

You must:

- train LoRA using the configuration supplied in the student template notebook;
- evaluate the LoRA-adapted model on `undrip_val.txt` and `economic_test.txt`;
- complete the LoRA row of the master results table;
- provide one generation comparison table after all three model states are available.

Required evidence:

- the LoRA row of the master results table completed;
- one completed generation comparison table using the fixed generation prompt.

Prompt	Pretrained output	Full fine-tuned output	LoRA-adapted output	One-sentence comparison
`Economic self-determination for First Nations peoples requires`				

Task 4 Interpretation and learning reflection (20%)

This task is about explanation and evidence of your individual learning.

Task 4A Technical interpretation (15%)

Answer the following questions in at most **300 words** total. Your answers must refer to your own master results table and generation comparison table.

1. Using your reported perplexity values, what do your results show about adaptation to `undrip_val.txt` versus transfer to `economic_test.txt`?
2. Using your reported trainable-parameter count, training time, perplexity values, and generation comparison, what trade-off do you observe between full fine-tuning and LoRA?
3. Based on your own results, which method would you recommend under limited compute for a quick domain-adaptation baseline, and why?

Generic answers that do not refer to your actual results will receive limited credit.

Task 4B Learning reflection (5%)

Answer the following question in at most **150 words**.

Over the 11 weeks of this unit, what is one intuition about deep learning that has changed, become clearer, or become more nuanced for you?

Your answer must include:

- one earlier intuition, assumption, or misconception you had;
- the concept, activity, assignment result, lecture, practical, or example that changed or refined it;
- what you now understand differently;
- how this new understanding would affect the way you approach a future deep learning problem.

This is not a general course review. It should be a specific reflection on your own learning.

This reflection is marked for specificity and evidence of personal learning, not for saying something impressive or positive about the unit.

Overall notebook quality and reproducibility (5%)

Across the whole submission, marks will also consider:

- correct section order and section headings;
- notebook runs end-to-end without errors;
- the three generated text files are submitted;
- the required preprocessing protocol is followed;
- the master results table is completed;
- the generation comparison table is completed;
- code outputs are cleaned;
- tables and figures are labelled clearly;
- the fixed generation prompt is used exactly;
- discussion is concise and relevant.

What good work looks like

A strong submission will usually show the following qualities:

- the notebook is easy to navigate;
- data preparation, baseline, full fine-tuning, and LoRA stages are directly comparable;
- the master results table is complete and internally consistent;
- perplexity is computed and interpreted correctly;
- interpretation is explicitly supported by the student's own reported numbers and generated examples;
- discussion distinguishes **in-domain adaptation** from **broader transfer**;
- the learning reflection is specific and connected to the student's own understanding;
- the student demonstrates understanding of why the protocol was controlled.

Common mistakes to avoid

- using a different train/validation split for the UNDRIP report;
- evaluating on a different Economic report text;
- changing the generated text files after reporting results;
- spending excessive effort on perfect document cleaning instead of following the fixed protocol;
- treating the preprocessing task as an open-ended data-engineering project;
- changing the base model partway through the assignment;
- changing the tokenizer or evaluation protocol across comparisons;
- using a different generation prompt from the one specified in the brief;
- treating training text performance as the only important result;
- writing long unstructured commentary instead of concise analysis;
- writing generic analysis that does not refer to the reported perplexity values, parameter counts, training time, or generation examples;
- making claims about improvement or transfer without pointing to evidence from the student's own results;
- writing a generic learning reflection that does not identify a concrete change in understanding;

- including many extra experiments that make the notebook harder to follow;

Student template

Use the provided template file `SIT744_Assignment4_student_template.ipynb`.

If you do not use the template, your notebook must follow the same section order and headings.

Final checklist

Before submitting, make sure that:

- the filenames are correct;
- `undrip_train.txt`, `undrip_val.txt`, and `economic_test.txt` are submitted;
- the notebook runs end-to-end;
- the data-preparation table is completed;
- the 80/20 UNDRIP split is followed;
- all modelling tasks use the text files created in Task 1;
- the master results table is completed;
- the training-loss curve or final training-loss record is included;
- the generation comparison table is completed;
- the fixed generation prompt is used exactly;
- perplexity is reported consistently;
- the technical interpretation is concise;
- the technical interpretation refers to your own master results table and generation comparison table;
- the learning reflection identifies a concrete change in understanding;
- code outputs are cleaned;
- cloud resources are shut down after use.

Assurance of Learning

To assure learning outcomes and maintain assessment integrity in the context of generative AI, the teaching team may request selected students to participate in a short **oral verification interview**. Selection may be **random and/or based on marker review**.

During the interview, students may be asked to explain the reasoning, implementation, evaluation choices, or code structure used in their submission. If a student is unable to demonstrate understanding consistent with the submitted work, the assessment may be subject to further review under the unit's academic integrity procedures.

These interviews form part of the unit's **Assurance of Learning** approach.

END OF ASSIGNMENT FOUR